

Vertex Plugin Manager Specification

Version: 1.0

Status: Draft

1. Objetivo

O Plugin Manager é o subsistema do Vertex Core responsável por gerenciar todo o ciclo de vida dos plugins.

Ele atua como a autoridade central para instalação, carregamento, execução, suspensão, atualização e remoção de plugins, garantindo que todos operem de forma segura, isolada e consistente.

O Plugin Manager não executa diretamente o código dos plugins; essa responsabilidade pertence ao Execution Runtime.

2. Responsabilidades

O Plugin Manager é responsável por:

- descobrir plugins instalados;
- instalar e remover plugins;
- validar manifestos;
- verificar assinaturas digitais;
- resolver dependências;
- controlar o ciclo de vida;
- registrar plugins no sistema;
- disponibilizar informações para outros subsistemas;
- coordenar atualizações de plugins;
- manter o catálogo de plugins instalados.

3. Arquitetura

Plugin Manager

- |— Plugin Registry
- |— Manifest Loader
- |— Plugin Validator
- |— Dependency Resolver
- |— Lifecycle Controller
- |— Plugin Loader

- |— Plugin Unloader
- |— Plugin Installer
- |— Plugin Remover
- |— Plugin Catalog
- |— Plugin Metrics
- |— Plugin State Manager

Cada componente possui uma responsabilidade específica.

4. Componentes

Plugin Registry

Mantém o registro de todos os plugins conhecidos.

Armazena:

- identificador;
- versão;
- estado;
- permissões;
- dependências;
- capabilities.

Manifest Loader

Carrega e interpreta o Manifest de cada plugin.

Plugin Validator

Verifica:

- assinatura;
- integridade;
- compatibilidade;
- permissões;
- estrutura.

Dependency Resolver

Resolve dependências entre plugins.

Também detecta:

- dependências ausentes;
- ciclos;
- incompatibilidades.

Lifecycle Controller

Coordena todas as transições do ciclo de vida.

Plugin Loader

Prepara um plugin para execução.

Inclui:

- carregamento de classes;
- criação do PluginContext;
- registro de capabilities;
- registro de eventos.

Plugin Unloader

Libera recursos utilizados por um plugin.

Plugin Installer

Realiza a instalação de novos plugins.

Plugin Remover

Executa a remoção segura de plugins.

Plugin Catalog

Fornece consultas sobre plugins instalados.

Plugin Metrics

Coleta métricas de gerenciamento.

Plugin State Manager

Mantém o estado atual de cada plugin.

5. Fluxos

Instalação

Plugin

↓

Manifest

↓

Validação

↓

Registro

↓

Dependências

↓

Instalação

↓

Pronto

Inicialização

Plugin



Load



PluginContext



Lifecycle



Running

Remoção

Plugin



Stop



Unload



Remoção



Registro Atualizado

Atualização

Nova Versão

↓

Validação

↓

Compatibilidade

↓

Substituição

↓

Migração

↓

Running

6. APIs

Registro

- registerPlugin()
- unregisterPlugin()

Instalação

- installPlugin()
- removePlugin()

Carregamento

- loadPlugin()
- unloadPlugin()

Ciclo de Vida

- startPlugin()
- stopPlugin()
- suspendPlugin()
- resumePlugin()
- restartPlugin()

Consulta

- getPlugin()
- getPlugins()
- getPluginState()
- getManifest()

Dependências

- resolveDependencies()
- pluginDependents()

7. Estados

Not Installed

↓

Installed

↓

Validated

↓

Loaded

↓

Initialized

↓

Running

↓

Suspended

↓

Hibernated

↓

Stopped

↓

Unloaded

↓

Removed

Estados de erro:

- Invalid;
- Failed;
- Recovery.

8. Políticas

Manifest obrigatório

Nenhum plugin pode existir sem um Manifest válido.

Assinatura obrigatória

Plugins devem possuir assinatura válida conforme a política de segurança.

Dependências

Todas as dependências devem estar resolvidas antes da inicialização.

Isolamento

Cada plugin executa isoladamente.

Uma falha não deve comprometer outros plugins.

Atualização segura

Atualizações devem preservar dados sempre que possível.

Rollback deve estar disponível quando suportado.

Descarregamento

Plugins inativos podem ser descarregados para economizar recursos.

9. Integrações

O Plugin Manager comunica-se com:

- Plugin Lifecycle Runtime;
- Execution Runtime;
- Scheduler;
- Resource Manager;
- Memory Management;
- Security Framework;
- Policy Engine;
- Storage Engine;
- Configuration System;
- Capability System;
- Event System;
- State Engine;
- UI Runtime Engine;
- Update System;
- Logging Framework;
- Plugin Inspector.

O Plugin Manager é o ponto central de gerenciamento, mas delega responsabilidades específicas aos demais subsistemas.

10. Métricas

O Plugin Manager registra:

- número de plugins instalados;
- número de plugins ativos;
- tempo médio de carregamento;
- tempo médio de descarregamento;
- tempo de inicialização;
- falhas de instalação;
- falhas de validação;
- falhas de dependência;
- tempo de resolução de dependências;
- quantidade de plugins suspensos;
- quantidade de plugins hibernados;
- reinicializações realizadas.

Essas métricas auxiliam diagnóstico e otimização da plataforma.

11. Exemplos

Instalação

1. Usuário instala um plugin.
2. Manifest é carregado.
3. Assinatura é validada.
4. Dependências são resolvidas.
5. Plugin é registrado.
6. Plugin fica disponível para carregamento.

Inicialização

1. O Boot Manager solicita a inicialização.
2. Plugin Loader prepara o ambiente.
3. PluginContext é criado.
4. Lifecycle Runtime executa a inicialização.
5. O plugin entra no estado Running.

Suspensão

1. O Resource Manager detecta baixa utilização.
2. Plugin Manager solicita suspensão.

3. Estado é preservado.
4. Recursos temporários são liberados.

Atualização

1. O Update System identifica uma nova versão.
2. O Plugin Manager interrompe o plugin.
3. A nova versão é instalada.
4. Dados são migrados, quando necessário.
5. O plugin é reiniciado.

Remoção

1. O usuário solicita a remoção.
2. O Plugin Manager verifica dependências.
3. O plugin é interrompido.
4. Recursos são liberados.
5. O registro é removido.

12. Regras Arquiteturais

- Apenas o Plugin Manager pode alterar o ciclo de vida de um plugin.
- Plugins nunca gerenciam diretamente outros plugins.
- Todo carregamento deve passar pelo Plugin Loader.
- Toda validação deve ocorrer antes da execução.
- O Plugin Manager nunca executa código de plugins diretamente; essa responsabilidade pertence ao Execution Runtime.
- Toda interação com plugins deve utilizar contratos definidos pela Plugin API.

13. Limitações

O Plugin Manager não é responsável por:

- executar tarefas dos plugins;
- renderizar interfaces;
- controlar threads;
- aplicar políticas de energia;
- fornecer armazenamento;
- implementar comunicação entre plugins.

Essas funções pertencem a outros subsistemas especializados.

14. Futuras Extensões

O Plugin Manager poderá evoluir para oferecer:

- carregamento incremental de plugins;
- pré-carregamento inteligente baseado no uso;
- perfis de inicialização;
- isolamento por processos quando necessário;
- gerenciamento de múltiplas versões do mesmo plugin para testes;
- hot reload para desenvolvimento.

15. Resumo

O Plugin Manager é o núcleo de gerenciamento do ecossistema de plugins do Vertex. Ele controla todo o ciclo de vida dos plugins, garante validação, isolamento e resolução de dependências, além de coordenar sua integração com os demais subsistemas do Core. Sua arquitetura modular permite que a plataforma permaneça segura, escalável e eficiente à medida que o número de plugins cresce.